

Schonfeld Strategic Advisors - FE COO Team

Quantitative Developer Case Study

Prompt

Build a "Fundamentals Tracker": a dashboard that tracks company fundamentals from SEC filings and market data, displays key metrics and charts, and answers natural-language questions spanning both the reported numbers and the narrative in the filings - for a fixed set of five companies.

Overview

This exercise mirrors the actual work of the role: taking a defined, business-adjacent problem and shipping a clean, useful tool that an investment professional would actually open - with an AI capability layered on top.

We are evaluating the **engineering rails** - how you ingest and integrate multiple data sources, model and serve the data through a working dashboard, and wire up an LLM to answer questions over it in a grounded way. A basic understanding of financial statements is useful context, but deep finance knowledge is not required. The financial concepts involved are deliberately basic (revenue, margins, growth) and the filing sections we ask about are the plain-English narrative parts, not technical accounting. Your job is to get the model the right data and keep its answers grounded in the source - the model handles interpretation.

There is no single "correct" build, and we care far more about *how you think* than feature count. Approach this as if you were building an internal tool that a Portfolio Manager or Analyst would actually use.

The Companies (fixed universe)

Build against these five tickers:

Ticker	Company
NVDA	NVIDIA
MSFT	Microsoft
AAPL	Apple
GOOGL	Alphabet
ETN	Eaton

Your design should **not hardcode** to these names - adding or swapping a ticker should be a configuration change, not a rewrite. We will grade against this fixed set, but generalization is a plus.

The Data (three required sources)

1. **SEC EDGAR - structured fundamentals (XBRL)**. The reported financial-statement facts available through EDGAR's XBRL company-facts data. This is the tracker's structured backbone.
2. **SEC EDGAR - filing narrative (text)**. The narrative sections of each company's **latest two annual 10-K filings: Item 1A "Risk Factors" and Item 7 "Management's Discussion & Analysis (MD&A)."** These are the readable, prose

sections of the filing - no accounting knowledge required to work with them. (How you extract and make this text queryable is your call - precise section parsing or full-text retrieval are both fine.)

3. **Market data - Yahoo Finance.** Daily/period prices for the five tickers (via `yfinance`, `stooq`, Alpha Vantage free tier, or equivalent). This is required because at least one derived metric must combine market data with EDGAR fundamentals (see below).

Optional / bonus: any freely available alternative dataset you think adds value. This is your space to be creative - not required, but a strong, well-justified addition is rewarded.

Public / free sources only. Starting points: [SEC EDGAR](#) (structured XBRL APIs at `data.sec.gov`; filing documents via the submissions history), and the market-data libraries above.

The Task

Build a small, runnable dashboard application that does the following:

1. **Ingest from all three sources.** A real ingestion pipeline that connects to and pulls from the live sources. This wiring is part of what we're evaluating.
 2. **Persist the ingested data** in a format of your choice - CSV files, SQLite, PostgreSQL, or whatever you are most comfortable with. The priority is clean, well-structured storage with a clear schema; the technology is your call. PostgreSQL is a bonus but not required. **In addition, ship a seed/snapshot** of the ingested data (committed files, a seed script, or a pre-populated volume) so the dashboard and Q&A layer can be run and tested against stored data **without depending on live API availability at review time.**
 3. **Wire up a backend** to serve the fundamentals, prices, and derived metrics to the dashboard. How you structure this is your call - a FastAPI service is a natural fit and is what we'd expect, but a lightweight alternative is acceptable if it serves the dashboard cleanly. Justify your choice briefly in the writeup.
 4. **Compute these metrics:**
 - Reported: revenue, net income, EPS, gross margin, operating margin (free cash flow optional)
 - Derived: year-over-year growth and margins
 - **At least one cross-source valuation multiple** that joins EDGAR + market data - e.g. trailing P/E (latest reported annual EPS × current price) or P/S. (Trailing only - no forward estimates.)
 - **At least one additional metric of your own choosing** that you think would be analytically meaningful to an equity investor. Justify your choice in the design writeup - this is deliberately open-ended and is part of what we are evaluating.
 5. **Add a natural-language / agentic layer** that answers questions spanning **both** the structured numbers and the filing narrative. At minimum the model must do more than free-text chat - we want tool use / function calling, structured outputs, retrieval, or an agent loop. The key behavior we're looking for is **routing**: the system deciding whether a question needs the structured numbers, the filing text, or both. The *form* is your choice (direct LLM API, agent framework, MCP server, etc.). **Be explicit in your writeup about where the LLM is - and isn't - in the path, and why.**
- No API key?** Several providers offer free tiers with no credit card required. Good starting points: [Google AI Studio](#) (Gemini 2.5 Flash, 500 req/day), [Groq](#) (Llama / DeepSeek-R1, 1M tokens/day), [OpenRouter](#) (35+ free models, single endpoint), and [GitHub Models](#) (GPT-4o and others, free for any GitHub user). Document which provider and model you used in the README.
6. **Build a dashboard UI** that serves as the primary interface for the tool. UI library or framework is your choice - justify it briefly in the writeup. The dashboard must include at minimum:
 - A **company selector** (the five tickers)
 - A **fundamentals table** showing the required metrics (including your own addition) for the selected company, with year-over-year comparisons

- A **price chart** for the selected company
- A **Q&A interface** for the natural-language layer

Additional panels, cross-company views, or visualisations of your own metric are welcome but not required.

7. **Containerize the full stack** via Docker Compose as a bonus. If you do, `docker compose up` must bring up the app, data store, and any other services in a single command. If you don't, the repo must include a `run.sh` bash script that installs dependencies and starts the full application in a single command, alongside a committed dependency file (`requirements.txt` or `pyproject.toml`). The LLM configuration (base URL, API key, model name) must be read from **environment variables**, never hardcoded - we run an LLM proxy and will point it at the model you specify.

Constraints

- **Stack:** Python is required. A FastAPI backend is expected but not mandatory - a lightweight alternative is fine if justified. Data persistence technology is your choice. UI framework is your choice. Docker Compose is a bonus (see above). AI provider, agent framework, retrieval approach, and all other libraries are your call.
- **AI-assisted development is expected.** Use Claude Code, Cursor, or whatever you normally work with. We assume you do, and we'll walk through your decisions in a follow-up session. Build it the way you'd actually build it.
- **LLM configuration:** make base URL, API key, and model name configurable via environment variables, and **document which provider and exact model(s) you used** in the README so we can test on a like-for-like basis.
- **Data availability:** ingestion must genuinely connect to the live sources, *and* you must persist a snapshot so everything else runs without live API dependency at review time.
- **Time:** we anticipate ~8–10 hours of work, and you'll have **72 hours** to submit. We want a working prototype, not a production system. Scope deliberately - knowing what to leave out is a signal we're looking for, and we'll ask what you cut and why.
- **Finance:** a basic understanding of financial statements is useful context for this exercise, but deep expertise is not required. The concepts involved (revenue, margins, growth, valuation multiples) are entry-level. Period alignment and messy data are expected - we're interested in how you handle them, not perfection.

Questions the Service Should Handle

Your service should be able to answer questions like these (we'll test with our own variants):

- *"What were NVDA's revenue and net income for the last three fiscal years?"* (structured)
- *"Which of the five companies had the highest gross margin last year?"* (derived, comparative)
- *"What is AAPL's trailing P/E right now?"* (cross-source join)
- *"What new risk factors did NVDA add in its latest 10-K versus the prior year?"* (filing text)
- *"How did MSFT's revenue grow last year, and what did management attribute it to?"* (cross-modal - numbers + MD&A)
- *"What is the company's forward guidance for next quarter?"* (out of scope - the system should decline or say it doesn't have that, not invent an answer)

Deliverables

1. A **GitHub repo** containing all code.
2. An **AI planning document** - the scoping or planning output you produced with your AI-assisted development tool before starting to build (e.g. the initial plan, architecture breakdown, or task list Claude Code / Cursor generated). Include this as a file in the repo (e.g. `PLAN.md`). We are not grading it - we want to see how you used AI as a thinking partner, not just a code generator.
3. A **README** that lets us run the full stack and open the dashboard in one command and includes:

- Required environment variables (including the LLM config) and how to point the service at our own model
- Which provider and exact model(s) you used
- 2–3 example interactions - both dashboard screenshots or descriptions and natural-language Q&A examples

4. A short design writeup (~1–2 pages) covering:

- Architecture and data model - what you built and why
- How you integrated the three sources (especially the cross-source join)
- Where you used the LLM, where you deliberately didn't, and the reasoning
- Your choice of backend, UI framework, and data persistence technology, and why
- Your additional analytical metric - what it is, why you picked it, and what it tells an investor
- Key tradeoffs, and what you deliberately left out given the time box

Evaluation Criteria

Dimension	What We're Looking For
Engineering Quality	Clean, idiomatic, well-structured Python; sensible project layout; tests, docs, and error handling proportional to a prototype
Data & Analytical Design	Thoughtful data model for time-series financial data (period alignment, handling messy or restated data); sound integration of multiple sources; clear separation of concerns (e.g. ingestion vs. serving)
AI Integration	The model is used where it earns its place - routing, tool use, structured outputs, or retrieval over naive prompting; answers stay grounded; awareness of failure modes and when <i>not</i> to use the model
Dashboard & Presentation Quality	A UI that an analyst would actually open; clear layout, readable data display, and a Q&A interface that feels usable; the additional metric is visualised or surfaced in a way that adds genuine insight
Product Judgment	A clear sense of the user and the problem; output that would actually be useful
Scope & Pragmatism	A working prototype with deliberate, well-reasoned cuts
Communication	Clear README and writeup; reproducible; easy to follow

Submission Instructions

- Name your files / repo using the format: `LastName_FirstName_QuantDev_CaseStudy`
- Please submit within **72 hours** of receiving the case to pgeoffroy@schonfeld.com, with the following language included:

I have completed the following Schonfeld Case Study solely using publicly available sources of information. Accordingly, I further confirm that I have not used any Material Non-Public Information (within the meaning of U.S. federal securities laws) and information that is subject to a duty or contractual restriction to a third party.

We look forward to reviewing your work. Good luck!

PJ Geoffroy

Schonfeld Strategic Advisors - Fundamental Equity Strategies